

PADRÃO REPOSITORY

Faculdade UNIDESC

Programação Para Web II

Profº Ruben Prado

PADRÃO REPOSITORY

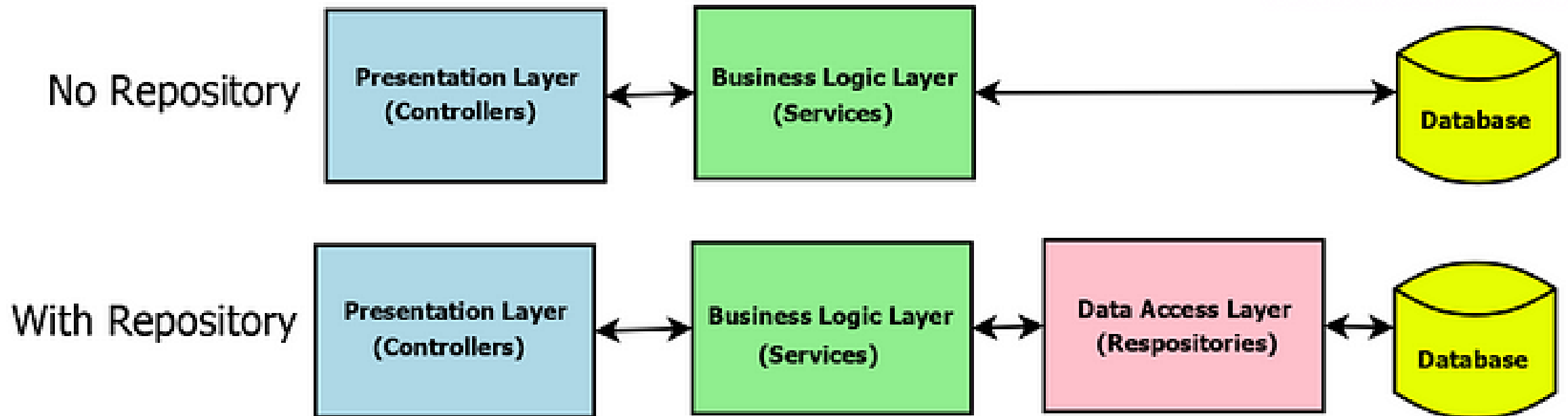
- O Padrão Repository é um padrão de design amplamente utilizado no desenvolvimento de aplicações web, incluindo aquelas construídas com ASP.NET Core MVC. Ele tem como objetivo principal abstrair a lógica de acesso a dados, separando-a da lógica de negócios da aplicação. Isso resulta em um código mais organizado, fácil de manter, testável e flexível para futuras mudanças.

PADRÃO REPOSITORY

O Padrão Repository funciona como uma camada intermediária entre a lógica de negócios da aplicação e a fonte de dados, como um banco de dados. Ele encapsula todas as operações de acesso a dados — como criar, ler, atualizar e deletar (CRUD) — em uma interface bem definida. Isso permite que a aplicação interaja com os dados de maneira consistente, sem se preocupar com os detalhes de implementação da persistência, como o tipo de banco de dados ou a tecnologia usada.

No contexto do ASP.NET Core MVC, o Repository é frequentemente utilizado para isolar a lógica de acesso a dados dos controladores e serviços, promovendo uma arquitetura mais limpa e modular.

Como Funciona o Padrão Repository em ASP.NET Core MVC?



Como Funciona o Padrão Repository em ASP.NET Core MVC?

1. Definição da Interface do Repository
2. Implementação Concreta
3. Injeção de Dependência
4. Uso nos Controladores

Definição da Interface do Repository

```
public interface ICategoriaRepository
{
    IEnumerable<Categoria> Categorias { get; }
}
```

```
public interface ILanchesRepository
{
    IEnumerable<Lanche> Lanches { get; }
    IEnumerable<Lanche> LanchesPreferidos { get; }
    Lanche GetLancheById(int lancheId);
}
```

- Crie uma pasta com o nome “Repositories”
- Dentro dela crie uma pasta com o nome “Interfaces”
- No exemplo acima, são criadas duas Interfaces, dentro da pasta Interfaces.

Implementação Concreta

```
public class CategoriaRepository : ICategoriaRepository
{
    private readonly AppDbContext _context;
    public CategoriaRepository(AppDbContext context)
    {
        _context = context;
    }
    public IEnumerable<Categoria> Categorias =>
        _context.Categorias;
}
```

Salve a Classe na pasta Repositories;

Implementação Concreta

```
public class LanchesRepository : ILanchesRepository
{
    private readonly ApplicationDbContext _context;

    public LanchesRepository(ApplicationDbContext context)
    {
        _context = context;
    }

    public IEnumerable<Lanche> Lanches =>
        _context.Lanches.Include(c => c.Categoria);

    public IEnumerable<Lanche> LanchesPreferidos =>
        _context.Lanches.Where(l =>
            l.IsLanchePreferido).Include(c =>
                c.Categoria);

    public Lanche GetLancheById(int lancheId)
    {
        return _context.Lanches.FirstOrDefault(l =>
            l.LancheId == lancheId);
    }
}
```

Salve a Classe na pasta Repositories;

Injeção de Dependência

```
using DegeLanches.Context;
using DegeLanches.Repositories;
using DegeLanches.Repositories.Interfaces;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddTransient<ICategoriaRepository, CategoriaRepository>();
builder.Services.AddTransient<ILancheRepository, LancheRepository>();

builder.Services.AddControllersWithViews();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production scenarios
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();
```

Add New Scaffolded Item

Installed

Common

API

Blazor

MVC

Controller

View

Razor Pages

Identity

Layout



MVC Controller - Empty



MVC Controller with read/write actions



MVC Controller with views, using Entity Framework

MVC Controller - Empty

by Microsoft
v1.0.0.0

An empty MVC controller.

Id: MvcControllerEmptyScaffolder

- Controller...
- Razor Component...
- New Item... Ctrl+Shift+A
- Existing Item... Shift+Alt+A
- New Scaffolded Item...
- New Folder
- Application Insights Telemetry...
- .NET Aspire Orchestrator Support...
- Container Orchestrator Support...
- Docker Support...
- Client-Side Library...
- New Azure WebJob Project
- Existing Project as Azure WebJob
- Class... Shift+Alt+C
- New EditorConfig

- View in Browser (Microsoft Edge)
- Browse With...
- Add
- Collapse All Descendants Ctrl+Left Arrow
- Scope to This
- New Solution Explorer View
- Exclude From Project
- Cut Ctrl+X
- Copy Ctrl+C
- Delete Del
- Rename F2
- Copy Full Path
- Open Folder in File Explorer
- Open in Terminal
- Properties Alt+Enter
- Ctrl+Unity Project... Solution Expl... GitHub Copil...
- Add to Source Control Select Repository

Add

Cancel

GitHub Copilot

Solution Explorer

Search Solution Explorer (Ctrl+ç)

◀ DegelLanches

▶ Connected Services

▶ Dependencies

▶ Properties

▶ wwwroot

◀ Context

▶ C# AppDbContext.cs

▶ Controllers

◀ Migrations

▶ C# 20250530183044_MigrationInitia

xtModelSnapshot.

del.cs

Repository.cs

Repository.cs

Repository.cs

Repository.cs

Repository.cs

Repository.cs

Repository.cs

Repository.cs

Repository.cs

Repository.cs

Repository.cs

Repository.cs

Repository.cs

Repository.cs

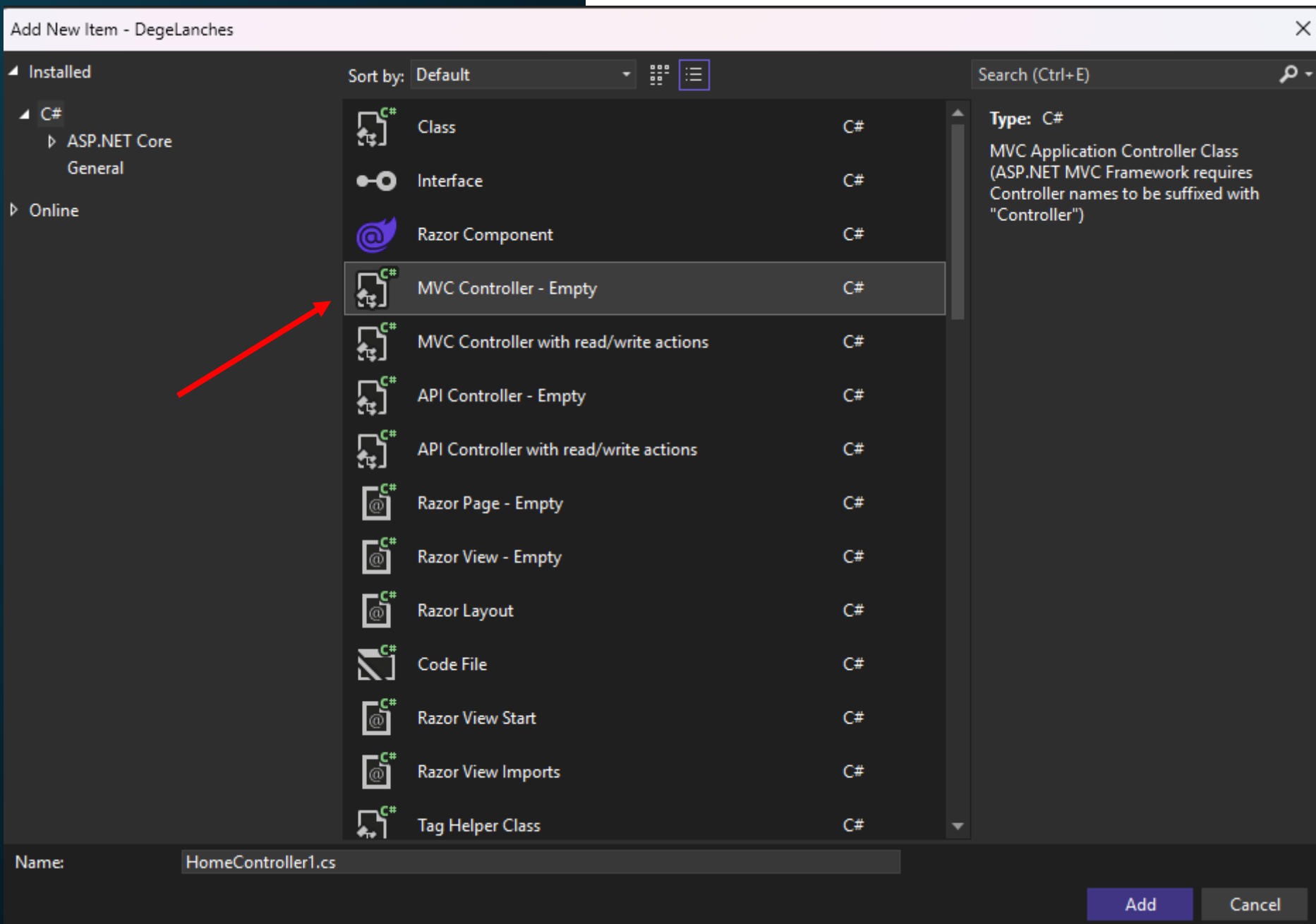
Repository.cs

Repository.cs

Repository.cs

Repository.cs

Repository.cs

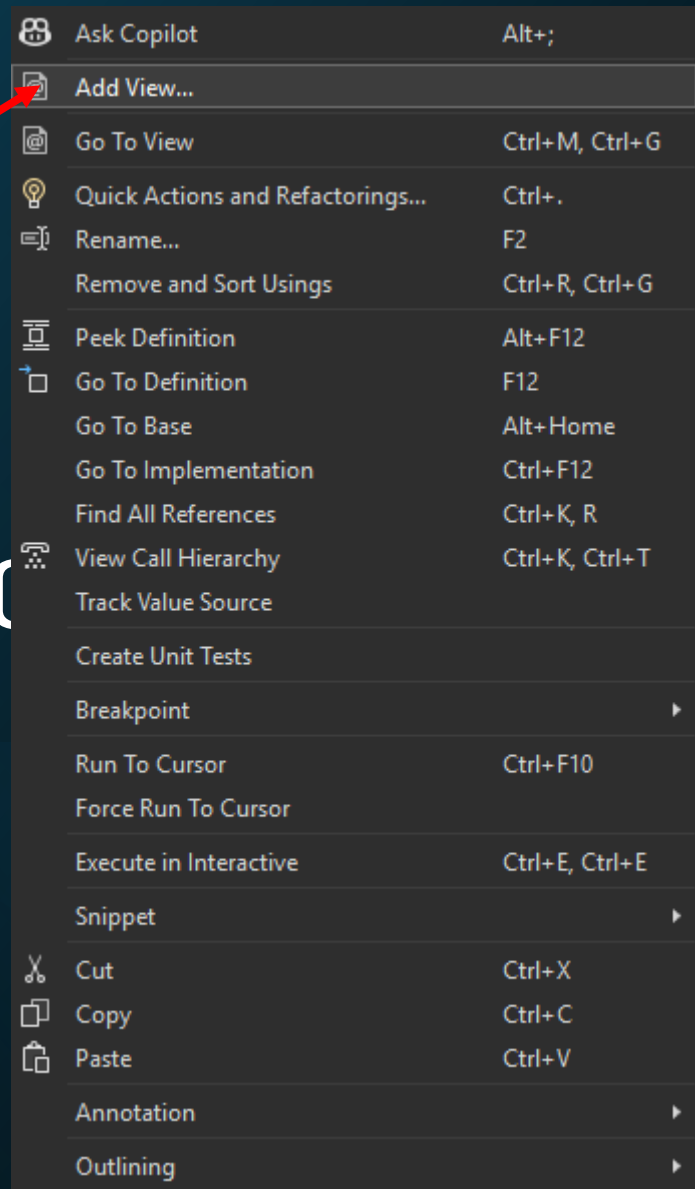


Uso nos Controladores

```
public class LancheController : Controller
{
    private readonly ILancheRepository
        _lancheRepository;

    public LancheController(ILancheRepository
        lancheRepository)
    {
        _lancheRepository = lancheRepository;
    }

    public IActionResult List()
    {
        var lanches = _lancheRepository.Lanches;
        return View(lanches);
    }
}
```



```
public class LancheController : Controller
{
    private readonly ILancheRepository
        _lancheRepository;

    public LancheController(ILancheRepository
        lancheRepository)
    {
        _lancheRepository = lancheRepository;
    }

    public IActionResult List()
    {
        var lanches = _lancheRepository.Lanches;
        return View(lanches);
    }
}
```

Add New Scaffolded Item

- Installed
 - Common
 - API
 - Blazor
 - MVC
 - Controller
 - View**
 - Razor Pages
 - Identity
 - Layout

 Razor View - Empty

 **Razor View**

Razor View
by Microsoft
v1.0.0.0
A Razor view with or without a strongly-typed Model
Id: RazorViewScaffolder



Add Razor View

View name

List

Template

List

Model class

Lanche (DegeLanches.Models)

DbContext class

AppDbContext (DegeLanches.Context)

Database provider

Configured from the selected DbContext

- Options
- ☐ Create as a partial view
 - ☐ Reference script libraries
 - ☒ Use a layout page

...

(Leave empty if it is set in a Razor _viewstart file)

Uso nos Controladores

```
public class LancheController : Controller
{
    private readonly ILancheRepository
        _lancheRepository;

    public LancheController(ILancheRepository
        lancheRepository)
    {
        _lancheRepository = lancheRepository;
    }

    public IActionResult List()
    {
        var lanches = _lancheRepository.Lanches;
        return View(lanches);
    }
}
```

REFERÊNCIAS

[Repository Design Pattern in ASP.NET Core MVC using EF Core - Dot Net Tutorials](#)

[Implementando o repositório e os padrões de unidade de trabalho em um aplicativo MVC ASP.NET \(9 de 10\) |](#)

[Microsoft Learn](#)

[Estrutura do projeto MVC do .NET Core: implementando um padrão genérico de serviço e repositório - Comunidade DEV](#)

[Padrão de repositório C# | Média](#)